

VantaDB

Plan Maestro de Redirección del Producto, Arquitectura y Ejecución

Fecha: 13 de abril de 2026

Documento de decisión, enfoque y ejecución para la siguiente etapa de VantaDB

Tesis central: VantaDB debe dejar de empujarse como una base de datos universal temprana y concentrarse en convertirse en un núcleo embebido de memoria persistente y retrieval híbrido, local-first, coherente y operable.

Fecha: 13 de abril de 2026

Autor: ChatGPT

Base documental considerada: Informe estratégico y técnico, PRD técnico + roadmap de 90 días + backlog, `todo.md`, `problemas\_encontrados\_en\_tests.md`, `vanta\_certification.json`, y el contexto acumulado del proyecto VantaDB.

0. Propósito del documento

Este documento existe para una sola cosa: **redireccionar VantaDB hacia un camino ejecutable, coherente y defendible.**

No está escrito para inflar la visión.

Está escrito para proteger el proyecto de tres riesgos que hoy son reales:

1. ****Dispersión estratégica****: intentar construir demasiadas cosas al mismo tiempo.
2. ****Deuda estructural prematura****: endurecer arquitectura y tooling antes de cerrar semántica del producto.
3. ****Autoengaño técnico****: confundir un motor prometedor con un producto listo.

El objetivo no es limitar tu ambición.

El objetivo es **convertir la ambición en una secuencia que sí pueda ejecutarse por un solo desarrollador sin romper el núcleo.**

1. Veredicto ejecutivo

1.1 Decisión principal

VantaDB no debe seguir tratándose como una “base de datos multimodelo universal” en esta etapa.

Debe redefinirse y ejecutarse como:

*****un núcleo embebido y local-first de memoria persistente y retrieval híbrido para agentes, herramientas locales y aplicaciones de conocimiento.*****

Ese es el posicionamiento correcto.

No porque sea más pequeño.

Porque es el único que hoy alinea:

- valor de producto,
- arquitectura viable,
- esfuerzo realista,
- ventaja competitiva,
- y capacidad de ejecución de un solo desarrollador.

1.2 Qué significa eso en la práctica

Tu ventaja inicial no debe ser:

- “guardo todo”
- “reemplazo varias categorías maduras”
- “soy embedded, server, multimodel, agent memory y enterprise a la vez”

Tu ventaja inicial debe ser:

- instalación simple,
- persistencia real,
- búsqueda vectorial + filtros,
- estructura documental útil,
- namespaces,
- rebuild reproducible,
- y una ruta clara hacia hybrid search.

1.3 Mi recomendación central

La dirección recomendada es inequívoca:

1. *****embedded-first*****
2. *****memoria persistente como producto*****

3. **retrieval híbrido como diferenciación**
4. **Fjall como backend principal del core**
5. **RocksDB como backend opcional o benchmark/escape hatch**
6. **índices como materializaciones derivadas**
7. **server mode como wrapper posterior**
8. **enterprise fuera del MVP**

2. Diagnóstico franco del proyecto

2.1 Lo que VantaDB sí tiene hoy

VantaDB ya tiene varias señales de proyecto serio:

- una tesis de mercado válida,
- un motor HNSW con validación interna,
- documentación estratégica más madura que muchos proyectos tempranos,
- una identidad técnica parcialmente definida,
- y una intuición correcta: **documento + embedding + metadata + relaciones** resuelven un dolor real cuando no están fragmentados.

Además, la certificación interna no es humo.

El stress protocol reporta:

- recall@10 de **0.9660** en ground truth 50K/128D,
- escalado validado 10K → 50K → 100K,
- crecimiento lineal de memoria de ~1060–1076 bytes/vector,
- persistencia round-trip sin pérdida de recall,
- y consistencia de grafo sin huérfanos.

Eso confirma que **hay un núcleo técnico rescatable y con valor real**.

2.2 Lo que VantaDB no tiene hoy

No tiene todavía un producto cerrado.

No tiene todavía una semántica multimodelo suficientemente definida.

No tiene todavía una política de mutación y coherencia que garantice que documento, embedding, filtros e índices no diverjan bajo cambios reales.

No tiene todavía hybrid search real de primer nivel.

No tiene todavía una instrumentación lo bastante confiable como para dirigir tuning serio.

No tiene todavía una frontera fuerte entre:

- core embebido,
- wrapper servidor,
- y ambición enterprise.

2.3 El problema principal no es el algoritmo

El problema principal **no** es HNSW.

No es RocksDB por sí solo.

No es Windows por sí solo.

No es el linker por sí solo.

El problema principal es **desalineación**.

Hoy el proyecto intenta sostener simultáneamente:

- motor embebido,
- modo servidor,
- memoria para agentes,
- base multimodelo general,
- Python in-process,
- despliegue Docker,
- narrativa local-first,
- y proyección enterprise.

Eso no es una estrategia.
Es demasiada superficie de ataque para tu etapa actual.

2.4 La lectura correcta de la situación

Tu proyecto no está fallando por falta de potencial.
Está en riesgo por exceso de amplitud.

Y esa clase de error mata proyectos buenos.

No porque el código sea malo.
Sino porque el fundador empieza a optimizar diez sistemas antes de tener un solo producto nítido.

3. Qué debe ser VantaDB de ahora en adelante

3.1 Definición oficial recomendada

VantaDB es un motor embebido de memoria persistente y retrieval híbrido para aplicaciones de conocimiento.

Traducción operativa

Debe permitir que un desarrollador pueda:

1. instalarlo sin infraestructura pesada,
2. guardar documentos/chunks con metadata,
3. asociar embeddings,
4. buscar por vector + filtros,
5. luego por texto + vector,
6. reiniciar sin perder coherencia,
7. exportar/importar,
8. reconstruir índices,
9. y operar todo localmente.

Eso sí es un producto.

3.2 Qué no debe ser en el MVP

Durante la primera etapa, VantaDB **no** debe asumirse como:

- base distribuida,
- plataforma HA,
- graph DB full-featured,
- motor enterprise con RBAC y auditoría,
- SQL completo,
- query language central del producto,
- data warehouse,
- ni runtime acoplado a LLMs dentro del core.

3.3 Posicionamiento correcto

La narrativa recomendada es:

“Memoria persistente embebida con búsqueda híbrida y filtros estructurados, para agentes y aplicaciones de conocimiento.”

Esa frase sí es defendible.

4. Hallazgos técnicos clave que condicionan la hoja de ruta

4.1 Hallazgo 1 — El stress protocol interno sirve, pero no basta

Los resultados internos son útiles para validar integridad y escalado del índice bajo datos sintéticos normalizados.

Eso es valioso.

Pero no debes vender eso como equivalencia competitiva externa.

Internamente te sirve para:

- regresiones,
- tuning,
- control de recall,
- persistencia,
- edge cases,
- y crecimiento de memoria.

Externamente todavía no es suficiente para compararte seriamente con motores maduros.

4.2 Hallazgo 2 — El benchmark competitivo con SIFT1M hoy no es decisión-grade

El documento de problemas detecta una limitación crítica:

- VantaDB usa **cosine similarity**
- SIFT1M usa ground truth **L2 / Euclidean**
- por tanto el recall competitivo actual es engañoso y no comparable.

Eso invalida el benchmark competitivo como señal de producto, no como señal de crash-resistance.

La conclusión es simple:

*****No uses hoy SIFT1M como argumento competitivo. Úsalo solo como disparador de la implementación de Euclidean.*****

4.3 Hallazgo 3 — La telemetría de memoria es un incidente de plataforma

Los logs reportan consumos incompatibles con el hardware perfilado.

Eso no es un detalle.

Cuando la telemetría de RAM es dudosa, cualquier decisión sobre:

- footprint,
- mmap,
- caches,
- compaction,
- benchmarks,
- sizing,
- y límites operativos

queda contaminada.

Mi posición aquí es dura:

hasta que arregles la telemetría, no deberías hablar de eficiencia de memoria con demasiada seguridad.

4.4 Hallazgo 4 — El mayor riesgo técnico no es velocidad, es coherencia

Tu riesgo dominante no es que una búsqueda tarde 4 ms más.

Tu riesgo dominante es devolver resultados con divergencia entre:

- documento canónico,
- embedding,
- payload/filter index,
- índice invertido,
- y relaciones.

En otras palabras:

tu principal problema no es ANN.

Tu principal problema es **gobierno de mutaciones y consistencia entre representaciones.**

4.5 Hallazgo 5 — El proyecto necesita separación de capas ya

La arquitectura objetivo correcta es:

1. ****Storage canónico****
2. ****Índices derivados****
3. ****Query planner****
4. ****Runtime embebido / SDK****
5. ****Wrapper servidor opcional****

Y la regla estructural es no negociable:

*****el core embebido no depende del server; el server envuelve al core.*****

5. Decisiones estratégicas que debes tomar y no volver a debatir cada semana

5.1 Decisión A — Embedded-first es política del producto

Esto debe quedar congelado.

No más ambigüedad entre:

- “somos como SQLite”
- “somos como una vector DB servida”
- “somos ambos al mismo tiempo”

Puedes soportar ambos modelos en el largo plazo, sí.

Pero la prioridad fundacional debe ser:

- ****core embebido****
- ****server wrapper después****

5.2 Decisión B — StorageBackend es obligatorio

No es opcional.

No es “nice to have”.

Es el punto de bifurcación que te permite dejar de reescribir el producto alrededor del backend actual.

Debes introducir:

- `StorageBackend`
- `FjallBackend`
- `RocksDbBackend` opcional
- `InMemoryBackend` para tests/dev

Sin esto, cada discusión sobre persistencia seguirá acoplada al backend concreto.

5.3 Decisión C — Fjall debe ser el backend principal del core

Mi recomendación es mantener la línea de tus propios documentos:

- ****Fjall como principal****
- ****RocksDB como opcional****

Razón real

No porque RocksDB sea peor en abstracto.

Sino porque hoy pagas mucha fricción de build, FFI y toolchain sin convertirla en ventaja entregada al usuario.

Tu problema actual no es falta de madurez industrial.

Tu problema actual es foco, velocidad de iteración y coherencia de producto.

5.4 Decisión D — El dato canónico manda; los índices obedecen

Esto tiene que volverse doctrina del proyecto.

Fuente de verdad

- documento

- metadata
- journal/WAL
- versión

Derivados reconstruibles

- ANN / HNSW
- índice invertido / BM25
- payload indexes
- caches

Si no congelas esta regla, vas a terminar persistiendo cosas en lugares conceptualmente incorrectos y luego reconstruyendo bugs imposibles de depurar.

5.5 Decisión E — La búsqueda híbrida no es extra; es la verdadera v1

Para el MVP vendible puedes vivir con:

- vector search + filtros

Pero para una v1 realmente diferenciada necesitas:

- índice invertido mínimo,
- BM25 o equivalente,
- RRF,
- payload indexes,
- planner básico.

Eso no es “decoración”.

Eso es lo que convierte el proyecto de una vector store embebida interesante en una memoria útil y más completa.

5.6 Decisión F — Enterprise queda explícitamente fuera del primer ciclo

No sigas dejando esa puerta entreabierta en la narrativa diaria.

Puedes mantener una visión enterprise.
No debes ejecutar como si ya estuvieras ahí.

6. Qué debes dejar de hacer de inmediato

6.1 Dejar de ampliar alcance

No abras nuevas líneas como prioridad:

- más modelos de datos,
- más integración con runtimes de LLM,
- más promesas cloud,
- más narrativa universal,
- más superficie de API sin semántica cerrada.

6.2 Dejar de tratar el benchmark competitivo actual como prueba de competitividad

Hasta implementar Euclidean, no sirve para compararte correctamente con SIFT1M.

6.3 Dejar de mezclar producto y demo

Una demo puede tolerar:

- sidecars raros,
- acoplamientos temporales,
- claims ambiguos,
- rutas de ejecución duales.

Un producto no.

6.4 Dejar de tratar el server mode como si fuera prioridad equivalente al core

No lo es.

No debe serlo.

No ahora.

6.5 Dejar de construir features sobre telemetría defectuosa

Primero arreglas observabilidad.

Después tuneas.

7. Qué debes empezar a hacer de inmediato

7.1 Congelar la definición del MVP

Necesitas un documento aprobado y operativo que diga con claridad:

Dentro del MVP

- embedded/local-first,
- documento + metadata + embeddings,
- vector search + filtros,
- namespaces,
- persistencia durable,
- export/import,
- rebuild ANN,
- SDK Python simple,
- CLI o UI local mínima.

Fuera del MVP

- multi-tenancy real,
- RBAC,
- HA,

- cloud gestionado,
- IQL como centro del producto,
- graph traversal complejo como prioridad comercial,
- integración profunda con pandas/Arrow,
- runtime LLM acoplado,
- replicación avanzada.

7.2 Definir el modelo canónico

Debe quedar explícito:

- identidad del documento,
- versión,
- estado actual,
- metadata tipada,
- embedding vigente,
- flags de reindexación,
- namespace,
- timestamps,
- parent_version_id,
- estado de consistencia,
- política de delete,
- y journal/recovery.

Sin ese modelo, todo lo demás es implementación sobre arena.

7.3 Escribir un protocolo de mutación

Debes definir, por escrito y luego en código, qué ocurre cuando:

1. entra un documento nuevo,
2. cambia el contenido,
3. cambia solo metadata,
4. cambia el embedding,
5. se borra lógicamente,
6. se reindexa,
7. el proceso cae entre write y indexación,
8. se reconstruye ANN,
9. se reconstruye invertido,

10. hay lectura concurrente durante reindexación.

Tu proyecto deja de ser “multimodelo bonito” y empieza a ser serio cuando esa tabla existe.

7.4 Aislar el core

Separar el core del server te da:

- builds más limpios,
- identidad más clara,
- menos acoplamiento,
- mejor DX,
- y menor superficie de regresión.

7.5 Replantear la estrategia de benchmark

La secuencia correcta es:

1. corregir telemetría
2. soportar Euclidean
3. rerun SIFT1M
4. comparar con referencia externa
5. medir flujo de memoria real del producto
6. no vender competitividad hasta que el benchmark sea semánticamente válido

8. Arquitectura objetivo recomendada

8.1 Capa 1 — Storage canónico

Responsabilidades:

- documento
- metadata

- versionado
- namespace
- journal/WAL
- snapshots
- replay
- tombstones / deletes lógicos
- checksums
- compaction policy

Principio

El storage canónico debe existir aunque mañana cambies por completo el índice ANN.

8.2 Capa 2 — Índices derivados

Responsabilidades:

- ANN / HNSW
- índice invertido / BM25
- payload indexes por campos
- caches opcionales

Principio

Todo índice debe ser reconstruible.

Ningún índice debe convertirse en fuente de verdad accidental.

8.3 Capa 3 — Query planner

Responsabilidades:

- decidir orden de ejecución,
- estimar selectividad,
- combinar vector + texto + filtros,
- escoger estrategia de ranking,
- exponer modo debug/trace.

En fase inicial

No necesitas planner complejo.

Necesitas uno **básico pero explícito**.

Ejemplo de decisiones iniciales:

- filtro altamente selectivo → primero payload/texto, luego vector
- consulta mixta → fusionar por RRF
- solo vector → ANN directo
- solo texto → BM25 directo

8.4 Capa 4 — Runtime embebido / SDK

Responsabilidades:

- API simple
- SDK Python
- CLI
- bindings futuros

Regla

La experiencia inicial debe parecer más cercana a SQLite que a desplegar una plataforma.

8.5 Capa 5 — Server wrapper opcional

Responsabilidades futuras:

- HTTP API
- auth
- observabilidad multiusuario
- límites
- administración

Regla

Wrapper, no corazón.

9. Modelo de datos recomendado

9.1 Unidad canónica sugerida

No sigas pensando solo en ``UnifiedNode`` como concepto técnico.

Empieza a pensar en una entidad canónica de memoria.

Campos mínimos recomendados

- ``namespace``
- ``key``
- ``document_id``
- ``version_id``
- ``parent_version_id``
- ``content``
- ``content_hash``
- ``metadata``
- ``embedding``
- ``embedding_status``
- ``text_index_status``
- ``vector_index_status``
- ``created_at``
- ``updated_at``
- ``deleted_at`` o ``is_deleted``
- ``consistency_state``

9.2 Estados de consistencia recomendados

Para evitar caos silencioso, usa un estado explícito:

- ``Consistent``
- ``PendingEmbedding``
- ``PendingVectorIndex``
- ``PendingTextIndex``
- ``PendingRebuild``

- `SoftDeleted`
- `CorruptNeedsRepair`

Eso te da trazabilidad operacional.

9.3 Política de mutación recomendada

Escritura base

1. escribir documento + metadata + versión + journal
2. marcar estados derivados como pendientes
3. generar o actualizar embedding
4. actualizar índices
5. marcar consistente
6. confirmar transacción lógica

Si hay crash a mitad

El recovery debe poder saber:

- qué estaba pendiente,
- qué necesita replay,
- qué necesita rebuild,
- qué versión es visible,
- y qué no debe exponerse como consistente.

10. Ruta técnica priorizada

10.1 Prioridad 0 — No negociables

1. congelar definición del MVP
2. introducir `StorageBackend`
3. crear perfil `dev-fast`
4. corregir telemetría de memoria
5. aislar core del server
6. prototipar Fjall

7. mantener RocksDB opcional
8. definir modelo canónico
9. namespaces first-class
10. CRUD mínimo
11. vector search + filtros
12. WAL + recovery
13. rebuild ANN
14. export/import
15. Python SDK mínimo usable
16. suite de crash/restart/recovery
17. README honesto
18. soporte Euclidean para benchmark competitivo válido

10.2 Prioridad 1 — Diferenciación real

1. índice invertido mínimo
2. BM25
3. búsqueda híbrida con RRF
4. payload indexes
5. versionado recuperable
6. protocolo de mutación documentado
7. métricas p95/p99
8. suite de brutalidad
9. inspección de índices y estado

10.3 Prioridad 2 — Después de que el núcleo aguante

1. planner adaptativo más serio
2. UI local mejor
3. wrapper server formal
4. auditabilidad profunda
5. cuotas
6. RBAC
7. multi-tenant
8. restore administrable
9. benchmark extendido y comparativas públicas
10. postura enterprise

11. Roadmap ejecutable de 90 días

Fase 0 — Congelación estratégica (Días 1-7)

Objetivo

Cerrar el alcance y eliminar ambigüedad de producto.

Entregables

- PRD aprobado
- definición oficial del MVP
- decisión formal embedded-first
- decisión formal Fjall principal / RocksDB opcional
- trait `StorageBackend`
- lista oficial de no-objetivos

Criterio de salida

- no se abren nuevas líneas de producto
- el backlog queda priorizado
- tú puedes explicar VantaDB en una frase sin contradicciones

Fase 1 — Estabilización del núcleo (Días 8-30)

Objetivo

Reducir fricción de build, sanear release engineering y preparar el desacoplamiento.

Entregables

- `StorageBackend`
- perfil `dev-fast`
- core separado del server
- saneamiento de packaging/naming que siga pendiente
- telemetría de memoria corregida
- prototipo Fjall
- RocksDB opcional detrás de interfaz
- soporte Euclidean en el índice
- rerun inicial de benchmark competitivo ya semánticamente correcto

Criterio de salida

- build reproducible
- telemetría creíble
- backend no acoplado al producto
- benchmark competitivo ya no contaminado por metric mismatch

Fase 2 — MVP funcional (Días 31-60)

Objetivo

Entregar memoria persistente usable.

Entregables

- modelo documento + metadata
- namespaces
- put/get/delete/list
- export/import
- vector search + filtros
- WAL / journal
- recovery / replay
- rebuild ANN
- Python SDK mínimo usable
- CLI mínima
- documentación operativa

Criterio de salida

- un usuario técnico instala, indexa, consulta y reinicia sin perder coherencia

Fase 3 — Diferenciación inicial (Días 61-90)

Objetivo

Pasar de vector memory a memoria híbrida.

Entregables

- índice invertido mínimo
- BM25 o equivalente
- RRF
- payload indexes

- versionado básico recuperable
- protocolo de mutación documentado
- métricas p95/p99
- suite de brutalidad
- benchmark del flujo de memoria
- decisión formal del wrapper server para etapa siguiente

Criterio de salida

- VantaDB deja de ser solo una base vectorial embebida
- y se convierte en una memoria persistente híbrida administrable

12. Plan de ejecución semanal detallado

Semana 1

- cerrar documento de definición del MVP
- congelar no-objetivos
- decidir oficialmente embedded-first
- aprobar estructura de capas
- abrir tickets P0

Semana 2

- introducir ``StorageBackend``
- crear ``InMemoryBackend``
- crear ``RocksDbBackend`` como adaptador actual
- diseñar superficie de ``FjallBackend``

Semana 3

- crear perfil ``dev-fast``
- medir build actual vs build dev-fast
- limpiar flags/perfiles que penalizan inner loop
- documentar comandos de desarrollo y CI

Semana 4

- separar crate/core del modo servidor
- mover wrappers server fuera del núcleo

- validar compilación independiente del core

Semana 5

- corregir telemetría de memoria
- separar claramente:
 - RAM del core
 - RAM del índice
 - page cache
 - mmap
 - buffers temporales
- crear benchmark pequeño de consistencia de medición

Semana 6

- prototipar `FjallBackend`
- validar CRUD mínimo
- mantener RocksDB como fallback opcional
- definir estrategia de migración interna

Semana 7

- implementar `DistanceMetric`
- añadir Euclidean
- ajustar heap/comparator
- versionar compatibilidad del índice
- rerun competitivo

Semana 8

- definir modelo canónico documento/version
- implementar namespaces first-class
- exponer `put/get/delete/list`

Semana 9

- WAL/journal básico
- recovery/replay
- crash tests controlados
- validar visibilidad consistente de versiones

Semana 10

- rebuild ANN determinista
- export/import
- SDK Python mínimo usable
- quickstart honesto

Semana 11

- índice invertido mínimo
- tokenización básica
- payload indexes iniciales

Semana 12

- RRF
- búsqueda híbrida
- protocolo de mutación documentado
- suite de brutalidad
- revisión de salida del ciclo de 90 días

13. Backlog ejecutable recomendado

13.1 P0 — No negociables

Producto

- Congelar definición del MVP
- Congelar no-objetivos
- Redefinir narrativa pública

Arquitectura

- `StorageBackend`
- core aislado
- modelo canónico
- protocolo de mutación
- namespaces first-class

Build y release

- perfil `dev-fast`
- telemetría corregida
- revisión final de naming/release/CI
- validación multiplataforma básica

Storage y durabilidad

- prototipo Fjall
- RocksDB opcional
- WAL
- recovery
- rebuild ANN
- export/import

Retrieval

- vector search + filtros
- Euclidean
- benchmark competitivo correcto

SDK / DX

- Python SDK mínimo
- CLI mínima
- README honesto
- quickstart reproducible

QA

- crash tests
- restart tests
- replay tests
- recovery tests

13.2 P1 — Muy importantes

- inverted index
- BM25
- RRF
- payload indexes
- versionado recuperable

- métricas p95/p99
- inspección de índices
- benchmark del flujo de memoria
- documentación del protocolo de coherencia

13.3 P2 — No tocar todavía como prioridad comercial

- planner adaptativo avanzado
- UI local rica
- wrapper server con gobierno
- audit log
- cuotas
- RBAC
- multi-tenancy
- HA / replicación avanzada
- cloud managed

14. Registro de riesgos y mitigaciones (FMEA simplificado)

Riesgo 1 — Divergencia entre documento e índices

Probabilidad

Alta

Impacto

Crítico

Síntoma

Resultados inconsistentes tras mutaciones o reinicios.

Mitigación

- source of truth canónica
- estados de indexación explícitos
- WAL + replay
- rebuild reproducible

- tests de crash durante indexación

Riesgo 2 — Acoplamiento estructural a backend equivocado

Probabilidad

Alta

Impacto

Alto

Síntoma

Cada cambio de storage obliga a reescribir partes del producto.

Mitigación

- `StorageBackend`
- pruebas contra backend en memoria
- Fjall prototipado temprano
- RocksDB como opcional

Riesgo 3 — Telemetría falsa guía decisiones incorrectas

Probabilidad

Alta

Impacto

Alto

Síntoma

Claims o tuning sobre consumo de memoria no reproducibles.

Mitigación

- rehacer instrumentación
- separar métricas por subsistema
- validar contra host real
- capturar medidas por escenario controlado

Riesgo 4 — Dualidad embedded/server rompe foco

Probabilidad

Alta

Impacto

Alto

Síntoma

builds lentos, arquitectura ambigua, marketing difuso.

Mitigación

- core sin deps server
- wrapper server posterior
- documentación que refleje el orden correcto

Riesgo 5 — Benchmark competitivo inválido produce falsas conclusiones

Probabilidad

Alta

Impacto

Medio-Alto

Síntoma

recall bajo en SIFT interpretado como debilidad del motor cuando es mismatch de métrica.

Mitigación

- Euclidean
- re-ejecución controlada
- comparativa solo cuando métrica y dataset sean compatibles

Riesgo 6 — Querer llegar a enterprise demasiado pronto

Probabilidad

Muy alta

Impacto

Crítico

Síntoma

roadmap inflado, features cosméticas, deuda operativa, producto no vendible.

Mitigación

- no-objetivos escritos
- criterio de salida del MVP estricto
- revisión quincenal de alcance

15. KPI y criterios de éxito

15.1 KPI de producto

- tiempo desde instalación hasta primera consulta útil
- número de pasos para indexar una fuente local
- tiempo de export/import
- tiempo de rebuild del índice
- tasa de éxito del flujo “instalar → indexar → buscar → reiniciar”

15.2 KPI técnicos

- p95/p99 de top-k con filtros
- throughput de ingesta por minuto
- recall sobre datasets representativos
- tiempo de recovery tras crash
- bytes por vector / chunk
- tamaño en disco por millón de registros
- consistencia de memoria reportada vs host real

15.3 Criterio real de salida del MVP

El MVP está listo cuando se cumplan simultáneamente estas condiciones:

1. el core compila y corre de forma estable en los entornos objetivo
2. la memoria es persistente y recuperable
3. existe búsqueda vectorial + filtros confiable
4. namespaces funcionan bien
5. el rebuild del índice existe y es usable
6. la observabilidad básica es creíble
7. export/import funciona
8. el SDK mínimo es usable
9. el producto se instala sin infraestructura pesada
10. y el valor básico no depende de otro motor externo

16. Organización del trabajo para un solo desarrollador

16.1 La ventaja real de ser un solo desarrollador

No es “hacer más rápido todo”.

Es poder sostener coherencia de visión y de diseño sin burocracia.

Pero eso solo es ventaja si conviertes esa libertad en disciplina.

16.2 Reglas operativas recomendadas

Regla 1

Solo una iniciativa P0 activa por vez.

Regla 2

Cada feature nueva debe responder:

- qué problema resuelve,
- qué deuda agrega,
- qué prueba la valida,
- y qué promesa de producto desbloquea.

Regla 3

No se fusiona nada crítico sin:

- criterio de aceptación,
- prueba automatizada,
- y nota de impacto arquitectónico.

Regla 4

Toda semana debe cerrar con una de estas tres cosas:

- menos ambigüedad,
- más durabilidad,
- o una experiencia usable mejor.

Regla 5

No optimices marketing, benchmark ni integración antes de cerrar semántica de mutación.

16.3 Cadencia recomendada

Lunes

decisión / arquitectura / prioridades

Martes a jueves

implementación profunda

Viernes

pruebas, documentación, limpieza, actualización de roadmap real

Sábado opcional

benchmark controlado o refactor pequeño, nunca expansión de alcance

16.4 Dashboard semanal mínimo

Debes revisar cada semana:

- build time local
- tests rotos
- estado de P0
- deuda nueva introducida
- bugs de consistencia
- progreso contra criterio de salida
- tareas que hay que matar

17. Lo que no debes hacer

1. no abrir nuevas promesas enterprise
2. no vender multimodelo universal todavía
3. no añadir otro motor de base de datos al stack
4. no acoplar LLM runtime al core
5. no usar el benchmark competitivo actual como validación pública
6. no dejar que el server wrapper dicte la arquitectura del core
7. no construir una UI rica antes de cerrar durabilidad y coherencia
8. no añadir más complejidad de consulta antes de tener source of truth clara
9. no persistir índices como si fueran el dato canónico
10. no perseguir feature parity con sistemas maduros desde el día uno

18. Decisiones concretas que yo tomaría hoy si dirigiera VantaDB

1. aprobar inmediatamente la redefinición del producto
2. fijar embedded-first como política
3. introducir `StorageBackend` esta semana
4. mover Fjall al frente del core
5. mantener RocksDB como escape hatch
6. tratar la telemetría rota como incidente crítico

7. implementar Euclidean antes de cualquier discurso competitivo
8. definir el modelo documento/version/estado antes de seguir ampliando la API
9. posponer server-first y enterprise
10. cerrar 90 días alrededor de memoria persistente híbrida, no de “multimodel database” genérica

19. Mi evaluación honesta de viabilidad

19.1 ¿Es viable el proyecto?

Sí.

19.2 ¿Es viable en la dirección equivocada?

No.

Si sigues intentando construir simultáneamente:

- motor general,
- server,
- memory system,
- hybrid DB,
- enterprise shell,
- y marketing de reemplazo universal,

vas a dispersarte.

19.3 ¿Es viable en la dirección correcta?

Sí, y bastante más.

Porque ahí entras en una categoría donde sí puedes diferenciarte:

- instalación simple,
- control local,
- persistencia real,
- búsqueda útil,

- estructura coherente,
- y poca fricción operativa.

20. Preguntas que conviene responder antes del siguiente ciclo, sin bloquear la ejecución

Estas preguntas no frenan el plan, pero sí mejoran la calidad del siguiente documento/iteración:

1. ¿Quieres que el primer usuario ideal sea más “builder de agentes” o más “herramientas documentales privadas”?
2. ¿Tu prioridad comercial futura está más cerca de SDK/librería o de producto instalable con interfaz?
3. ¿Quieres que el versionado sea obligatorio desde el MVP o controlado por flags/política?
4. ¿Qué nivel de compatibilidad Windows/Linux es realmente no negociable para ti en la primera venta/uso?
5. ¿Prefieres que la CLI sea el plano principal de administración o que exista una UI local mínima desde antes?

21. Siguiendo paso inmediato recomendado

En vez de abrir nuevas discusiones de alto nivel, yo haría esto en orden:

Próximas 24 horas

- aprobar formalmente la redefinición del producto
- aprobar el alcance del MVP
- aprobar la política embedded-first
- abrir backlog P0 definitivo

Próximas 72 horas

- diseñar `StorageBackend`
- definir modelo canónico
- auditar telemetría de memoria
- decidir estrategia concreta de migración Fjall

Próxima semana

- ejecutar Fase 0 completa
- arrancar Fase 1 sin abrir nuevas líneas de producto

22. Cierre

VantaDB no necesita más ambición en este momento.

Necesita **más filo**.

La oportunidad real no está en que digas que haces todo.

Está en que construyas una pieza de infraestructura que haga **muy bien** un problema real:

memoria persistente embebida + retrieval híbrido + estructura coherente + baja fricción.

Eso sí tiene mercado.

Eso sí tiene narrativa.

Eso sí tiene arquitectura defendible.

Y eso sí puede ser ejecutado por un solo desarrollador con criterio fuerte.

Tu limitación no es ser uno solo.

Tu limitación sería insistir en cargar cinco productos dentro del mismo código base antes de haber cerrado uno.

No hagas eso.

Construye el núcleo correcto.

Hazlo confiable.

Hazlo simple de instalar.

Hazlo consistente.

Hazlo híbrido en el orden correcto.

Y solo después expándelo.

Anexo A — Síntesis operativa de prioridades

P0 inmediato

- MVP scope
- StorageBackend
- dev-fast
- telemetría
- core separado
- Fjall prototipo
- Euclidean
- modelo canónico
- namespaces
- CRUD
- WAL/recovery
- rebuild ANN
- export/import
- SDK mínimo

P1 después

- inverted index
- BM25
- RRF
- payload indexes
- versionado recuperable
- métricas p95/p99
- suite brutalidad

P2 después de aguantar

- planner adaptativo serio
- UI rica
- wrapper server fuerte
- enterprise governance

Anexo B — Frase de producto recomendada

VantaDB es un motor embebido de memoria persistente y retrieval híbrido para agentes y aplicaciones de conocimiento, diseñado para operar localmente con baja fricción y un modelo de datos coherente.

Anexo C — Regla de oro de arquitectura

Documento + metadata + journal son la verdad.

ANN, BM25 y payload indexes son derivados reconstruibles.

El server envuelve al core. Nunca al revés.